



$$A = \begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{pmatrix}$$

COMPUTER GRAPHICS: GEOMETRY TRANSFORMATION

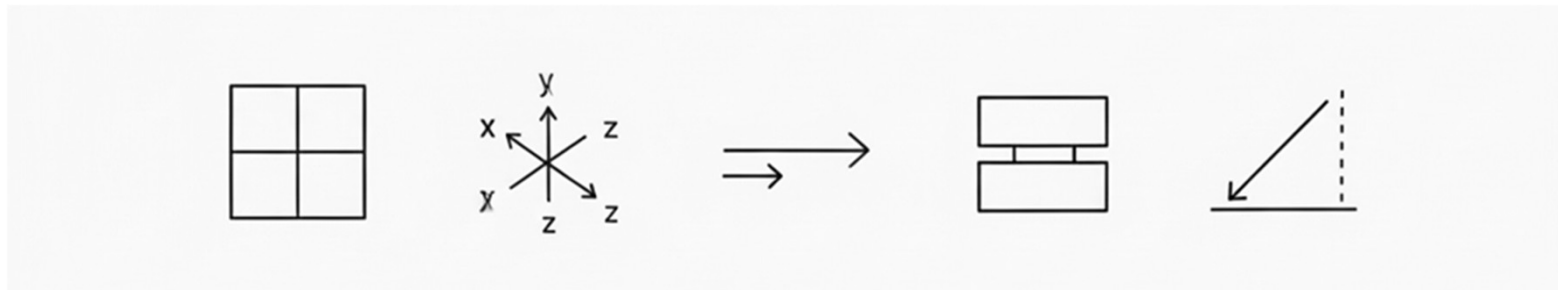
Lecturer: Giáp Văn Tiến Ph.D.
tiengv@vnu.edu.vn +84865276899
Faculty of engineering and technology

Lecturer: Giáp Văn Tiến Ph.D.
tiengv@vnu.edu.vn +84865276899
Faculty of engineering and technology

$$(l_2, o) = \begin{pmatrix} 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix}$$

Agenda and Intended Learning Outcomes

- 2D vs 3D transformations: linear, affine, and coordinate frames
- Translation and homogeneous coordinates (matrix form in 3D)
- Composition, decomposition (TRS), and inverse matrices
- Normals vs arbitrary vectors; correct normal transformation for lighting
- 50-minute guided practice + homework assignment

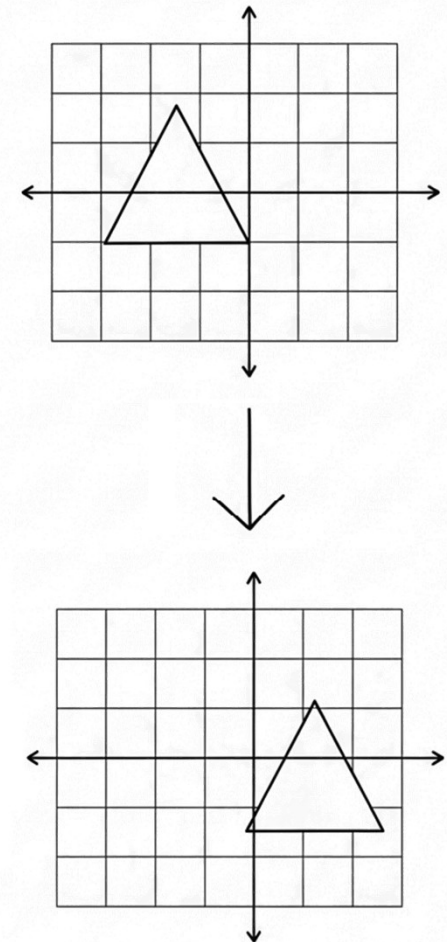


What Is a Geometric Transformation?

- **A function** that maps points/vectors from one space to another

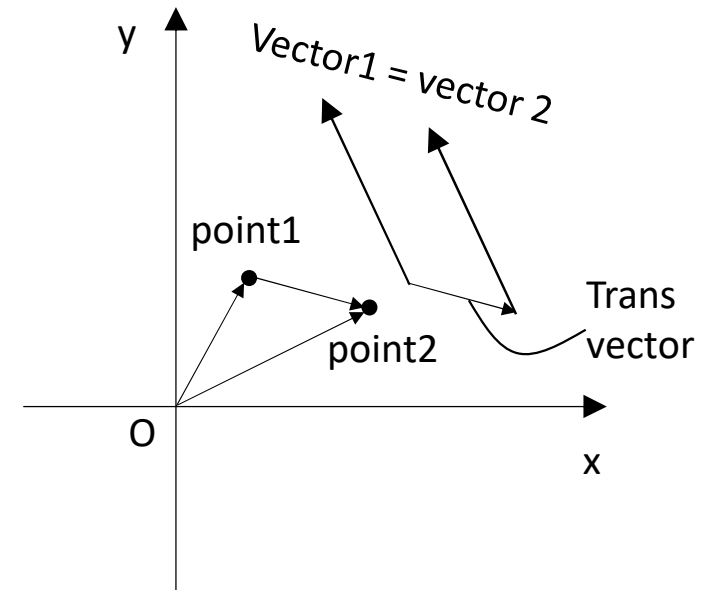
$$T(\mathbf{v}) = A\mathbf{v} \quad \left\{ \begin{array}{l} \mathbf{v} = \text{vector} \\ A = \text{transformation matrix} \\ \quad (3 \times 3 \text{ or } 4 \times 4) \end{array} \right. \quad \mathbf{v} = \begin{bmatrix} x \\ y \\ w \end{bmatrix} \text{ or } \begin{bmatrix} x \\ y \\ z \\ w \end{bmatrix} \quad \begin{array}{l} w=0 \text{ vector} \\ w=1 \text{ position} \end{array}$$

- Used to position objects (model), camera (view), and projection to the screen
- 2D/3D transformations share the same concept: multiply by matrices
- Key idea: choose the right space and the correct order of operations



Points vs Vectors and Coordinates

- **Point:** position in space (affected by translation);
- **Vector:** direction/offset (not translated)
- A **coordinate** frame is an origin plus a basis (three perpendicular axes in 3D)
- **Homogeneous** coordinates add w to unify affine transforms in one matrix multiply
- $w=1$ for points, $w=0$ for directions; translation lives in the last column of a 4×4
- This distinction is essential for correct motion and lighting calculations



```
// 4x4 affine transform (homogeneous)
glm::mat4 A = glm::mat4(1.0f);
A[3] = glm::vec4(tx, ty, tz, 1.0f); // translation column

glm::vec4 p2 = A * glm::vec4(p, 1.0f); // point or position
glm::vec4 v2 = A * glm::vec4(v, 0.0f); // vector or direction
```

$$v = \begin{bmatrix} x \\ y \\ w \end{bmatrix}$$

$$\text{pos1} = \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

$$\text{pos2} = \begin{bmatrix} x + dx \\ y + dy \\ 1 \end{bmatrix}$$

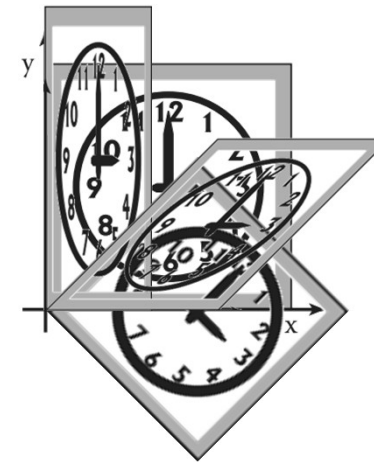
2D Linear Transformations and translation

- Linear transform in 2D: $\mathbf{v}' = A \mathbf{v}$ (A is 2×2) — preserves the origin
- Scaling, rotation, shear are linear;
- translation is not linear: $\mathbf{v}' = \mathbf{v} + \mathbf{dv}$
- Use 3×3 matrices for 2D affine transforms ($x, y, 1$)
- Geometric meaning: columns of A are where basis vectors end up

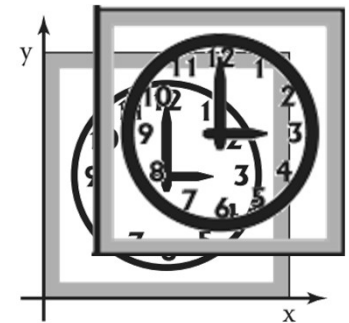
```
// 2D linear (use mat2)
glm::mat2 A(1.0f);
// apply to direction vectors
glm::vec2 v2 = A * v;
```

```
// 2D affine (use mat3 + homogeneous)
glm::mat3 H(1.0f);
glm::vec3 p2 = H * glm::vec3(p, 1.0f);
```

Linear transform



translation



Linear transform + translation = Affine transform

$$\mathbf{v}' = A\mathbf{v} + \mathbf{dv} = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} dx \\ dy \end{bmatrix} = \underbrace{\begin{bmatrix} a_{11} & a_{12} & dx \\ a_{21} & a_{22} & dy \\ 0 & 0 & 1 \end{bmatrix}}_M \cdot \begin{bmatrix} x \\ y \\ w \end{bmatrix}$$

homogeneous
 $w = 0$ vector
 $w = 1$ position

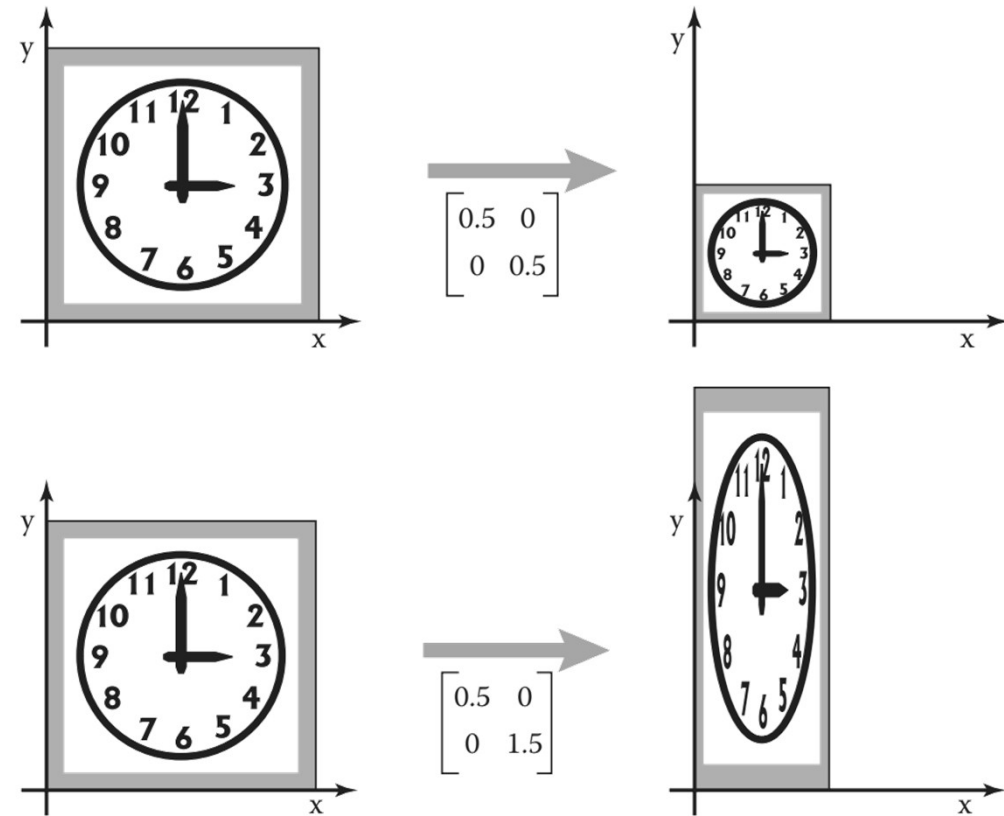
2D Scaling (Linear)

- Uniform scale: same factor in x and y;
- Non-uniform: different factors
- Non-uniform scaling changes angles; it also affects normals in 3D lighting
- *Scaling around a point* requires translate → scale → translate back

$$A = \begin{bmatrix} a_{11} & 0 \\ 0 & a_{22} \end{bmatrix}$$

```
//Affine transform
glm::mat3 T1 = glm::translate(glm::mat3(1.0f), -center);
glm::mat3 S  = glm::scale(glm::mat3(1.0f), glm::vec2(a11, a22));
glm::mat3 T2 = glm::translate(glm::mat3(1.0f), center);

glm::mat3 H = T2 * S * T1; // scale about 'center'
```



2D Rotation (Linear)

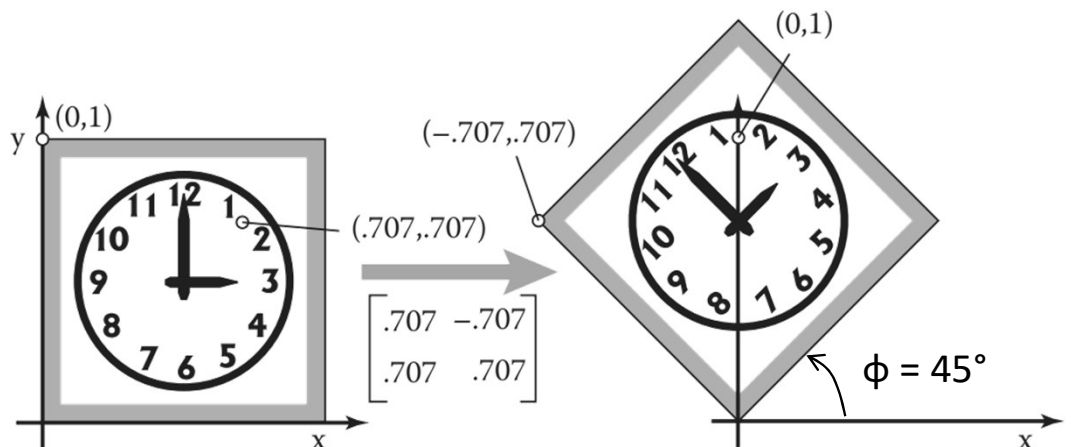
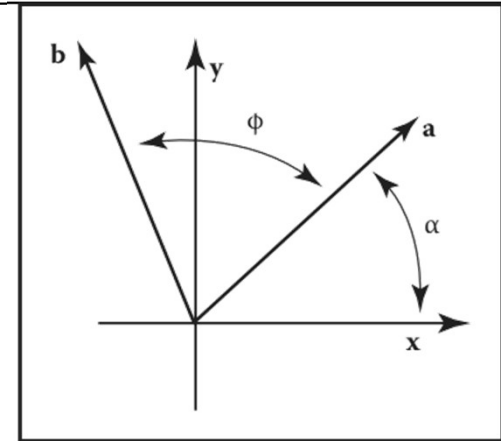
- Rotation preserves lengths and angles (rigid motion), $\det(A)=1$
- Direction depends on coordinate system and right-hand rule conventions
- Rotate about an arbitrary point: translate $\rightarrow$ rotate $\rightarrow$ translate back

Rotate from a $\rightarrow$ b:

$$A = \begin{bmatrix} \cos \phi & -\sin \phi \\ \sin \phi & \cos \phi \end{bmatrix}$$

```
float theta = glm::radians(30.0f);
glm::mat2 R(
    cos(theta), -sin(theta),
    sin(theta),  cos(theta)
);

glm::vec2 v2 = R * v;
```



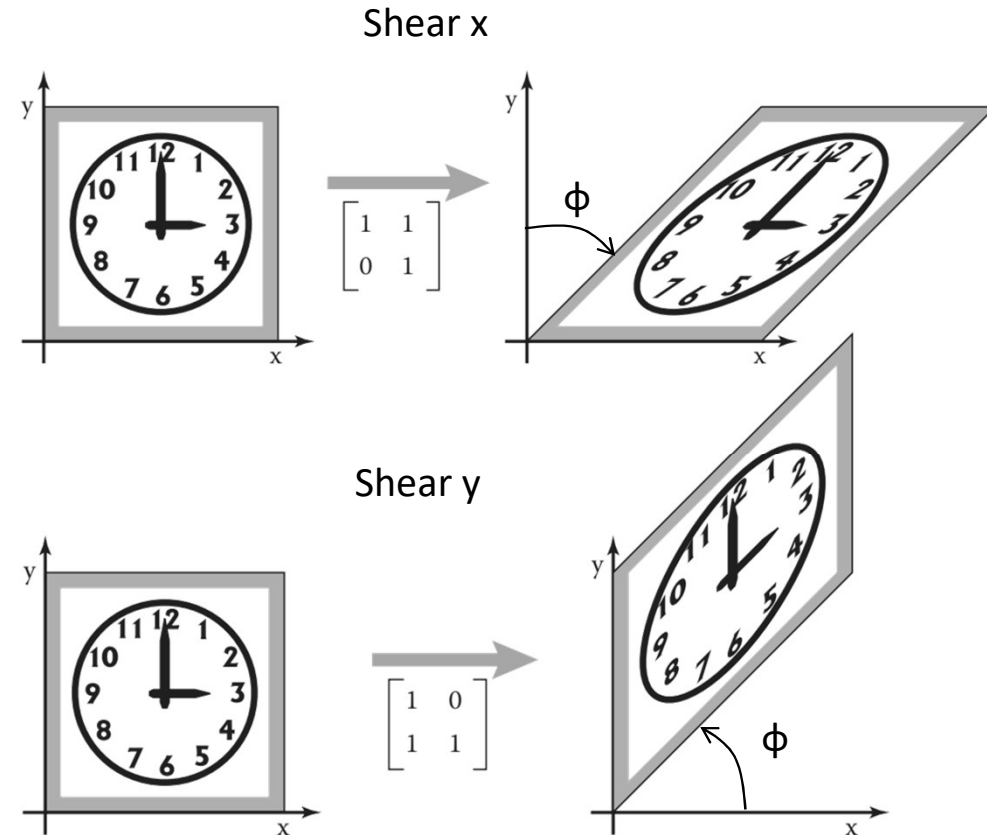
2D Shear (Linear)

- Shear shifts one axis proportionally to the other (parallelogram effect)
- Shear preserves area only when $\det(A)=1$; otherwise, it scales area
- Often used in font/italic effects and simple deformation

$$A_{shearX} = \begin{bmatrix} 1 & \tan \phi \\ 0 & 1 \end{bmatrix} \quad A_{shearY} = \begin{bmatrix} 1 & 0 \\ \tan \phi & 1 \end{bmatrix}$$

```
float k = 0.6f;
glm::mat2 Shx(
    1.0f, k,
    0.0f, 1.0f
);

glm::vec2 p2 = Shx * p;
```



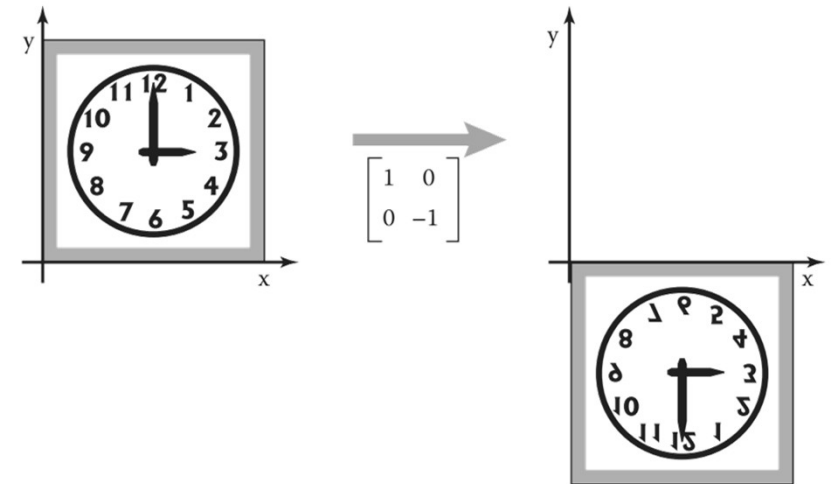
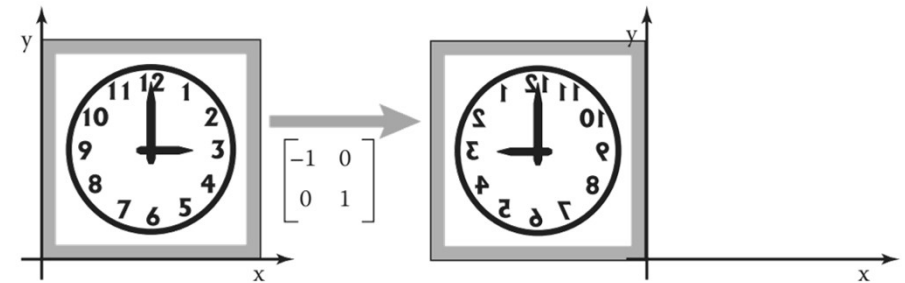
2D Reflection

- reflect a vector across either of the coordinate axes by using a scale with one negative scale factor

$$A_{ReflectX} = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} \quad A_{ReflectY} = \begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix}$$

$$\text{Det}(A_{\text{reflect}}) = -1$$

$$A_{ReflectXY} = \begin{bmatrix} -1 & 0 \\ 0 & -1 \end{bmatrix}$$



3D Rotation: Axes, Handedness, and Intuition

- 3D rotations can be around X/Y/Z axes, or around an arbitrary unit axis
- Right-hand rule: curl fingers from +X to +Y gives +Z direction
- OpenGL/GLM: use `glm::rotate(angle, axis)` for stable axis-angle rotation

$$\text{rotateX}(\phi) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \phi & -\sin \phi \\ 0 & \sin \phi & \cos \phi \end{bmatrix} \quad \text{rotateY}(\phi) = \begin{bmatrix} \cos \phi & 0 & \sin \phi \\ 0 & 1 & 0 \\ -\sin \phi & 0 & \cos \phi \end{bmatrix}$$

Homework: find matrix and try rotating around an arbitrary vector, quaternion

```
glm::mat4 R = glm::rotate(glm::mat4(1.0f),  
    glm::radians(45.0f),  
    glm::normalize(glm::vec3(1, 1, 0))  
);
```



$$\text{shear X} = \begin{bmatrix} 1 & dy & dz \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

shear Y = ?

3D Affine Transform Matrix (4×4 Block Form)

- Affine in 3D: $\mathbf{p}' = \mathbf{A} \mathbf{p} + \mathbf{t}$ ($\mathbf{A}$ is 3×3, $\mathbf{t}$ is 3×1)
- Homogeneous matrix: $[[\mathbf{A}, \mathbf{t}], [0 \ 0 \ 0, 1]]$
- Enables chaining by matrix multiplication (composition)

$$\mathbf{v}' = \mathbf{A}\mathbf{v} + \mathbf{d}\mathbf{v} = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ z \end{bmatrix} + \begin{bmatrix} dx \\ dy \\ dz \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ z \\ w \end{bmatrix}$$

homogeneous
w = 0 vector
w = 1 position

```
glm::mat4 M(1.0f);  
M = glm::translate(M, t);  
M = glm::rotate(M, angle, axis);  
M = glm::scale(M, s); // M = T * R * S (GLM post-multiply)
```

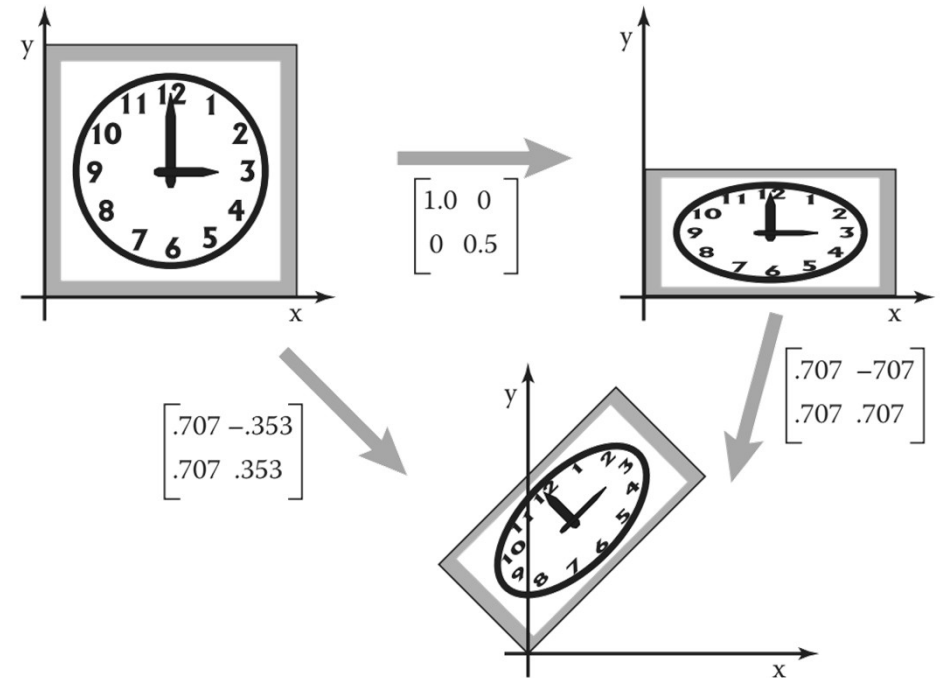
```
glm::mat4 M = glm::mat4(  
    1.0f, 0.0f, 0.0f, 0.0f, // Column 0  
    0.0f, 1.0f, 0.0f, 0.0f, // Column 1  
    0.0f, 0.0f, 1.0f, 0.0f, // Column 2  
    10.0f, 20.0f, 30.0f, 1.0f // Column 3 (translation)  
);
```

Composition: Order Matters

- Compose transforms by multiplication: $A = A_2 \cdot A_1$
- Rightmost matrix applies first
- Rotate then translate $\neq$ translate then rotate
- Debug tip: test with a single point (1,0,0) and see

$$v' = A_3 \cdot A_2 \cdot A_1 \cdot v$$

```
glm::mat4 A = glm::rotate(glm::mat4(1.0f), a, axis);  
glm::mat4 B = glm::translate(glm::mat4(1.0f), t);  
  
// Column-vector convention: rightmost first  
glm::vec4 p1 = (B * A) * glm::vec4(p, 1.0f); // rotate,  
then translate  
glm::vec4 p2 = (A * B) * glm::vec4(p, 1.0f); // translate,  
then rotate (orbit)
```



$$\begin{bmatrix} .707 & -.353 \\ .707 & .353 \end{bmatrix} = \begin{bmatrix} .707 & -.707 \\ .707 & .707 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 \\ 0 & .5 \end{bmatrix}$$

Decomposition: TRS (Translation–Rotation–Scale)

- Many model matrices are built as TRS: $M = T \cdot R \cdot S$
- Decomposing M helps edit position, orientation, scale separately
- Not every matrix decomposes cleanly (shear + non-uniform scale can mix)
- Practical approach: store T , R , S explicitly and build M when needed

$$v' = T \cdot R \cdot S \cdot v$$

```
glm::vec3 Tpos(1.0f, 0.0f, -3.0f);
glm::vec3 Sscale(1.2f, 0.8f, 1.0f);
float angle = glm::radians(20.0f);

glm::mat4 M = glm::translate(glm::mat4(1.0f), Tpos)
    * glm::rotate(glm::mat4(1.0f), angle,
glm::vec3(0,1,0))
    * glm::scale(glm::mat4(1.0f), Sscale);
```

glm function for decomposing matrix in
glm/gtx/matrix_decompose.hpp

GLM_FUNC_DECL bool glm::decompose()

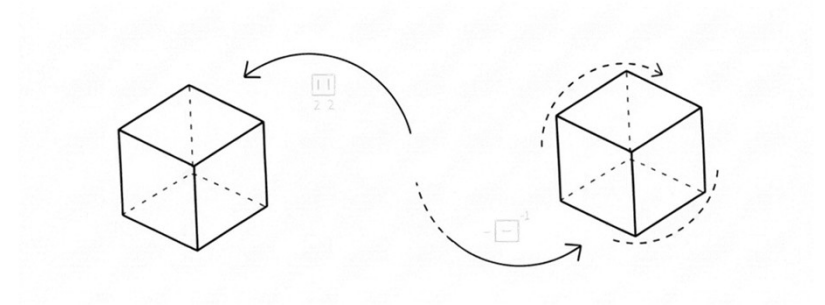
```
#include <glm/gtx/matrix_decompose.hpp>
bool glm::decompose(
    glm::mat4 const& modelMatrix,
    glm::vec3& scale,
    glm::quat& orientation,
    glm::vec3& translation,
    glm::vec3& skew,
    glm::vec4& perspective
);
```

Input: modelMatrix

Outputs (to address): vector scale, translation, skew, perspective

Inverse Transform: “Undo” a Transformation

- If M is invertible, M^{-1} maps back: $p = M^{-1} p'$
- Inverses reverse order: $(B \cdot A)^{-1} = A^{-1} \cdot B^{-1}$
- Inverse of translation: negate t ;
- inverse of rotation: transpose (for pure rotation)
- Used for coordinate transforms: world $\rightarrow$ local, world $\rightarrow$ view, etc.



```
glm::mat4 M = ...;
glm::mat4 Minv = glm::inverse(M);

// Reverse order example
glm::mat4 C = B * A;
glm::mat4 Cinv = glm::inverse(A) * glm::inverse(B);
```

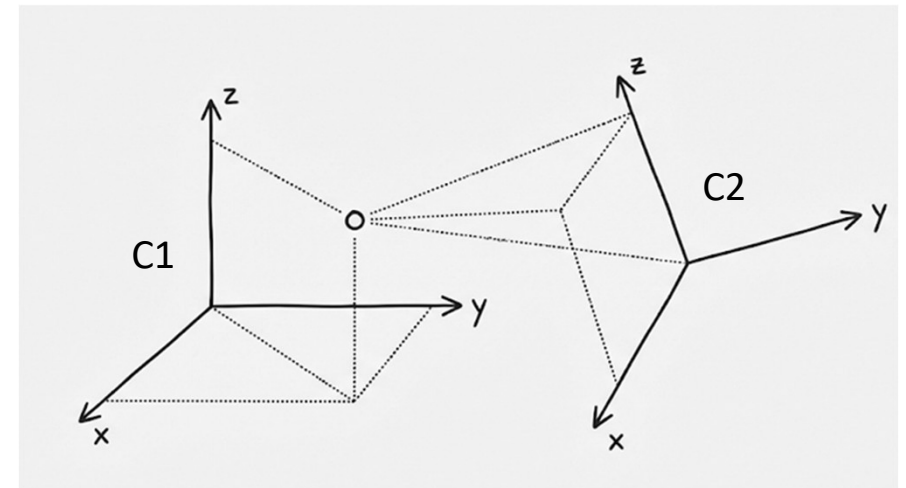
Change of Basis (Coordinate Transformation)

- A coordinate frame = origin + basis vectors (i, j, k)
- Object transform: move the object in world space
- Coordinate transform: re-express the same point in another frame
- Common graphics chain: Local $\rightarrow$ World $\rightarrow$ View (Camera) $\rightarrow$ Clip

$$C_2 = MC_1$$

$$\mathbf{v}_{C2} = \mathbf{M}^{-1} \mathbf{v}_{C1}$$

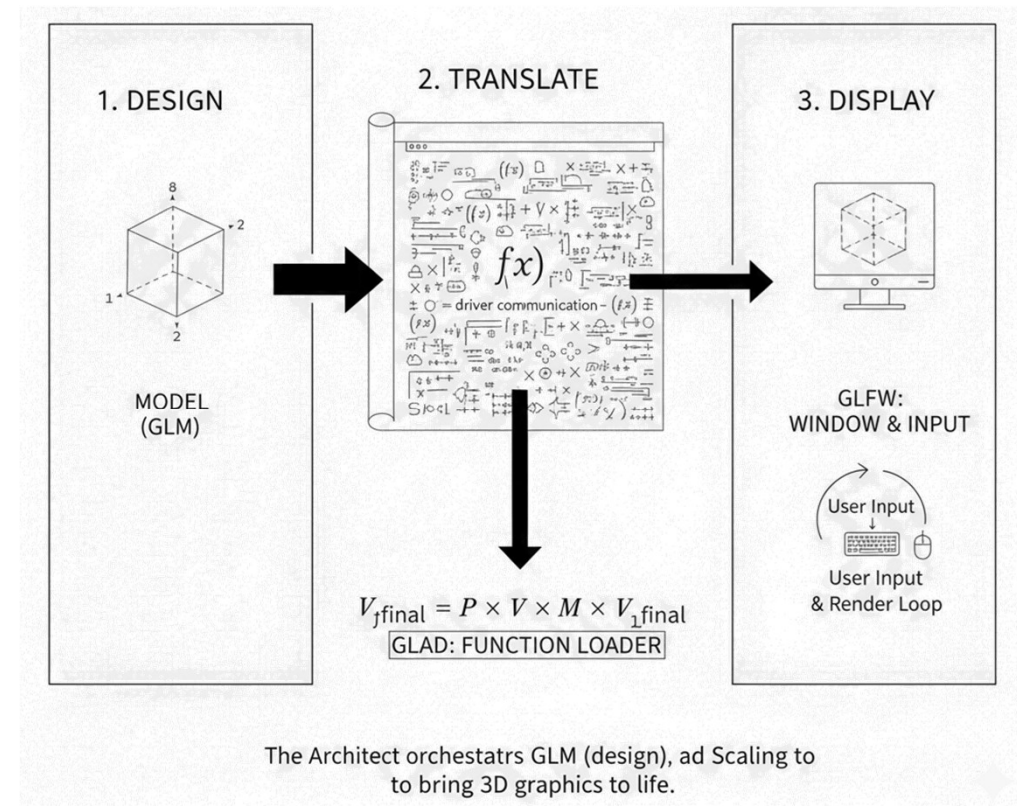
```
// Local-to-World (object transform)
glm::mat4 M = ...;
// World-to-View (camera transform)
glm::mat4 V = glm::lookAt(eye, target, up);
// View-to-Clip (projection)
glm::mat4 P = glm::perspective(fovy, aspect,
nearZ, farZ);
```



OpenGL + C/C++

- OpenGL is a graphical API, platform independent
- OpenGL contains over 200 functions :Primitive functions, Attribute functions, Viewing functions

Library	Purpose	Real-world Analogy
GLFW	Window & Input	The Gallery Building where the art is shown.
GLAD	Driver Communication	The Translator who speaks the GPU's language.
GLM	Linear Algebra	The Architect calculating the dimensions.
stb	Image Loading	The Crate that brings the paintings into the room.



OpenGL Transform Pipeline (MVP)

- Vertex positions are transformed:
$$\text{clipPos} = P \cdot V \cdot M \cdot \text{localPos}$$
- After perspective divide: NDC $\rightarrow$ viewport mapping to screen pixels
- Passing affine transforming matrix to shader
- Keep transforms explicit to avoid confusion: name matrices clearly
- Progressive debugging: start with M only, then add V, then add P

CPU (C++)

- └─ compute matrices (GLM)
- └─ send uMVP uniform
 - └─ GPU vertex shader
 - └─ transform vertices
 - └─ rasterization

```
void WINAPI glDrawArrays(  
    GLenum mode,  
    GLint first,  
    GLsizei count  
);  
  
GL_POINTS, GL_LINE_STRIP,  
GL_LINE_LOOP, GL_LINES,  
GL_TRIANGLE_STRIP,  
GL_TRIANGLE_FAN, GL_TRIANGLES,  
GL_QUAD_STRIP, GL_QUADS, and  
GL_POLYGON.
```

Program structure

```
// CPU side (*.cpp file)  
//create vertices upload to buffer  
float vertices[] = {  
    -0.6f, -0.4f, 0.0f, //col 1  
    0.6f, -0.4f, 0.0f, //col 2  
    0.0f, 0.6f, 0.0f //col 3  
};  
{buffer see cpp file, init() function}  
  
//create transformation matrix  
glm::mat4 M(1.0f);  
M = glm::translate(M, t);  
  
//send data to GPU & call draw function  
shader.use();  
shader.setMat4("uModel", M);  
glBindVertexArray(m_VAO);  
glDrawArrays(GL_TRIANGLES, 0, 3);  
  
// Vertex shader (*.vert file)  
layout (location = 0) in vec3 aPos;  
uniform mat4 uModel;  
void main(){  
    gl_Position = uModel*vec4(aPos,1.0);  
}
```

GLM Essentials: Building mat4 Transforms

- GLM uses column-major matrices; multiplication order assumes column vectors
- Start with identity (`mat4(1.0f)`); then apply translate/rotate/scale
- Prefer **radians** (`glm::radians`) and **normalized axes** for rotation
- Always print/inspect matrices during debugging (especially order)

```
1 #include <glm/glm.hpp>
2 #include <glm/gtc/matrix_transform.hpp>
3 #include <glad/glad.h>
4
5 glm::mat4 M(1.0f);
6 glm::vec3 pos(1.0f, 0.5f, 0.0f);
7
```

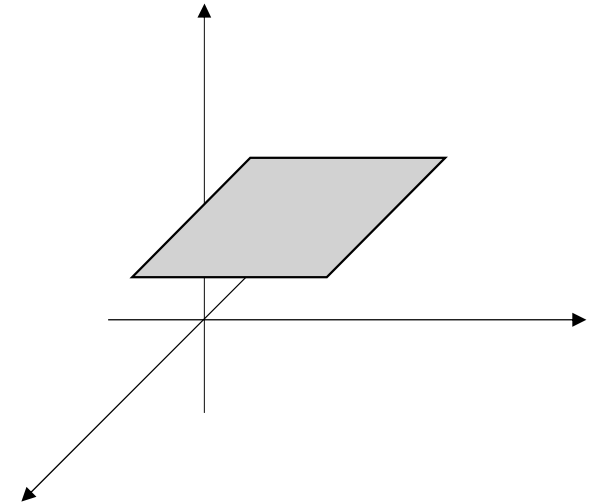
```
glm::mat4 M(1.0f);
M = glm::translate(M, t);
M = glm::rotate(M, angleRad, axisUnit);
M = glm::scale(M, s);
```

```
// Equivalent (clearer): M = T * R * S;
```

```
glm::mat4 mat = glm::mat4(
    1.0f, 0.0f, 0.0f, 0.0f, // Column 0
    0.0f, 1.0f, 0.0f, 0.0f, // Column 1
    0.0f, 0.0f, 1.0f, 0.0f, // Column 2
    10.0f, 20.0f, 30.0f, 1.0f // Column 3
    (translation) );
//modify mat4 value
model[3][0] = 5.0f; // Row 0 of Column 3
```

Implementing 2D Transforms in OpenGL

- You can render 2D using the same 3D pipeline (z=0 plane)
- Use an orthographic projection for pixel-like 2D control
- Represent 2D affine transforms with mat3 or mat4 (recommended: mat4)
- Progressive steps: render shape → apply scale → apply rotation → apply translation



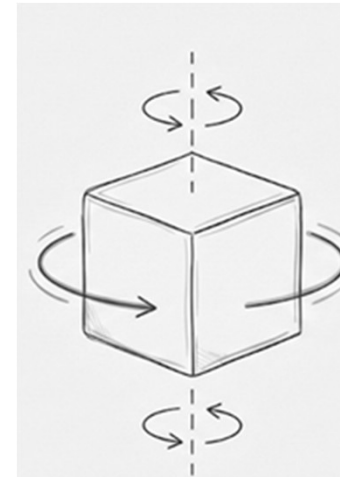
```
glm::mat4 P = glm::ortho(-1.0f, 1.0f, -1.0f, 1.0f);
glm::mat4 V = glm::mat4(1.0f); // no camera for 2D
glm::mat4 M = glm::translate(glm::mat4(1.0f), glm::vec3(tx, ty, 0.0f))
    * glm::rotate(glm::mat4(1.0f), a, glm::vec3(0,0,1))
    * glm::scale(glm::mat4(1.0f), glm::vec3(sx, sy, 1.0f));
```

Rotation: Pivot, Orbit, and Local Spin

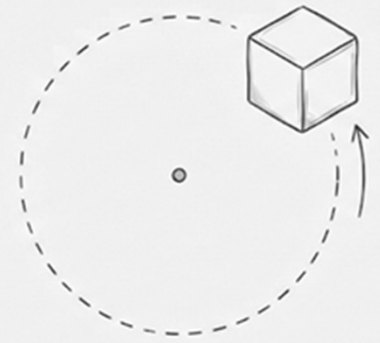
- Rotation is not only “spin in place”: orbiting is rotation about a pivot in world space
- Orbit pattern: $T(\text{pivot}) \cdot R(\theta) \cdot T(-\text{pivot})$ (changes the rotation origin)
- Local spin pattern: $T(\text{position}) \cdot R(\theta) \cdot S$ (spins around the object origin)
- Key idea: same matrices, different meaning because order changes

```
glm::vec3 pivot(0.0f, 0.0f, 0.0f);  
float theta = timeSeconds;  
  
glm::mat4 M = glm::translate(glm::mat4(1.0f), pivot)  
              * glm::rotate(glm::mat4(1.0f), theta,  
glm::vec3(0,1,0))  
              * glm::translate(glm::mat4(1.0f), -pivot);
```

spin



orbit

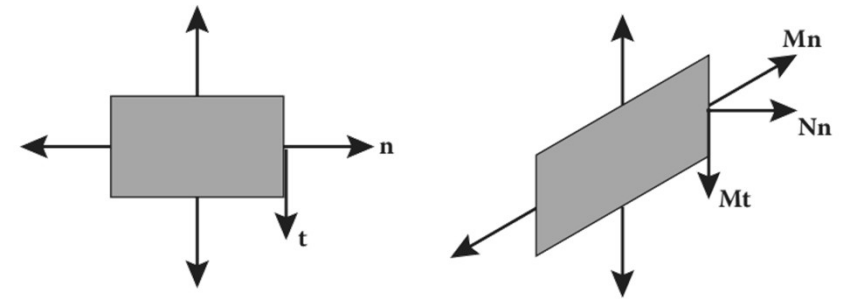


Normals Are Not “Just Vectors” (When Scaling/Shear Exists)

- Normals encode surface orientation for lighting (perpendicular to tangent plane)
- Applying the same matrix as positions can break perpendicularity under non-uniform scale/shear
- Goal: keep $n \perp$ tangent after transform
- Solution: transform normals with inverse-transpose of the linear part
- Translation is handled separately

```
// CPU (recommended)
glm::mat3 A = glm::mat3(M);
glm::mat3 N = glm::transpose(glm::inverse(A));
shader.setMat3("uNormalMat", N);

// GLSL
uniform mat3 uNormalMat;
vec3 nWorld = normalize(uNormalMat * aNormal);
```



Transform vector/vertex

$$v' = A \cdot v$$

Transform normal vector

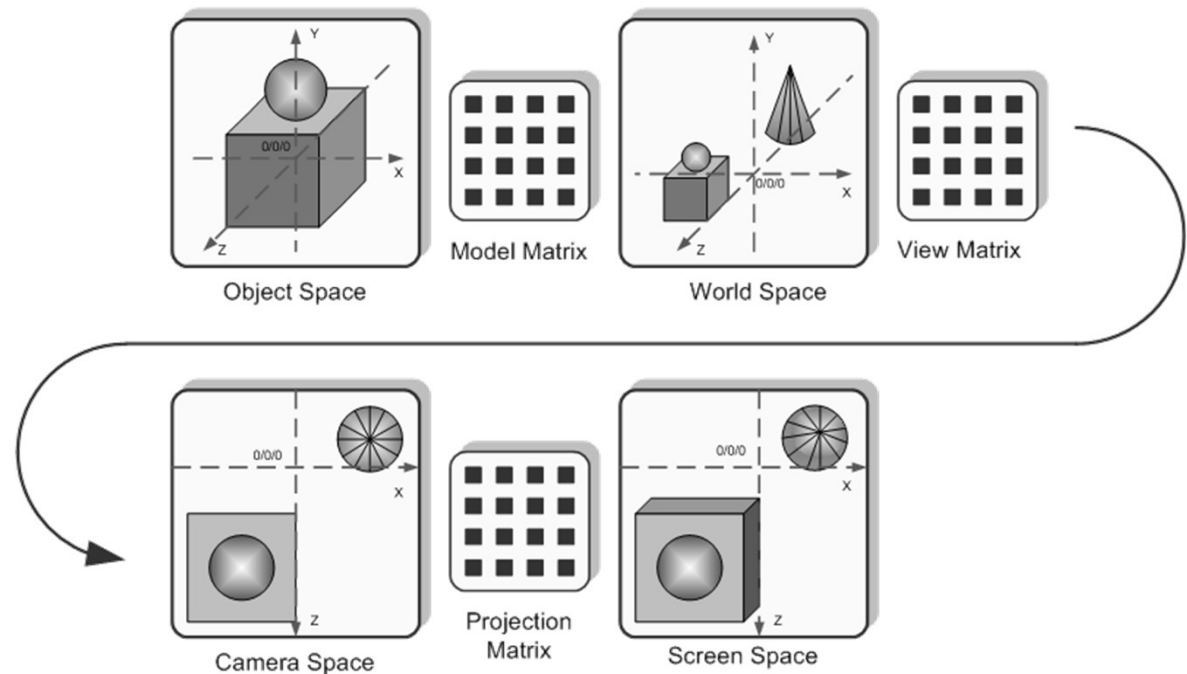
$$n' = (A^{-1})^T \cdot n$$
$$n' = \frac{n'}{|n'|}$$

Coordinate Transform in Practice: Local $\rightarrow$ World $\rightarrow$ View

- Local $\rightarrow$ World: model matrix M places object into the scene
- World $\rightarrow$ View: view matrix V expresses coordinates in camera space
- Coordinate transform uses inverse when switching description (world $\leftrightarrow$ local)
- Debug approach: visualize axes for each frame

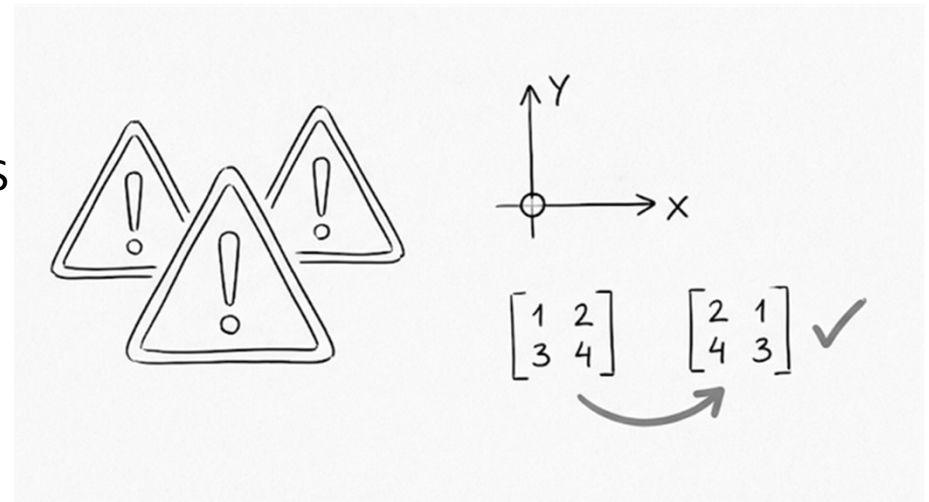
```
// Local to World  
vec4 pW = M * vec4(pL, 1.0);  
// World to View  
vec4 pV = V * pW;  
// View to Clip  
vec4 pC = P * pV;
```

```
// World to Local (coordinate change)  
vec4 pL2 = inverse(M) * pW;
```



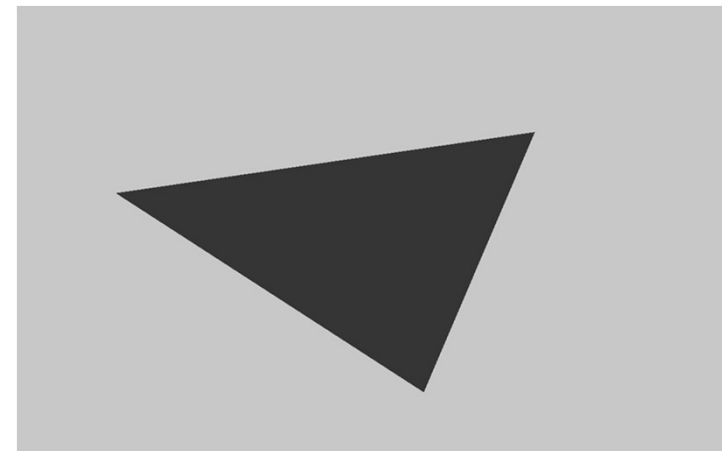
Common Pitfalls (and How to Avoid Them)

- Degrees vs radians: standardize on radians inside the program
- Order confusion: write transforms in a consistent TRS convention
- Mixing spaces: track whether vectors are in local, world, or view space
- Non-uniform scale affects normals: use correct normal matrix when lighting



Guided Practice (50 minutes): “Transform Lab”

- Goal: render a Triangle and interactively apply TRS + coordinate transforms
- You will implement: model matrix updates, composition, inverse, and normal matrix
- Tools: OpenGL + GLFW/GLAD + GLM (starter project provided)
- Deliverable: short demo + code commit + screenshots



Practice Steps 1–3: Baseline + 2D/3D Linear Transforms

- Step 1 (5m): compile & run starter; confirm triangle renders with flat color
- Step 2 (8m): add axis lines (X/Y/Z) to debug orientation
- Step 3 (7m): implement scale + rotation (linear part) using GLM
- Checkpoint: triangle rotates about its local origin; scaling changes size
- Step 4 (10m): add translation; implement orbit around pivot using composition
- Step 5 (8m): compute and use inverse to transform a point world→local (debug)

```
// Step 3: build model matrix (linear part)
glm::mat4 M(1.0f);
M = glm::rotate(M, rotY, glm::vec3(0,1,0));
M = glm::rotate(M, rotX, glm::vec3(1,0,0));
M = glm::scale(M, glm::vec3(sx, sy, sz));
shader.setMat4("uModel", M);
```

```
// Step 4: affine + composition (orbit)
M = glm::translate(glm::mat4(1.0f), pivot)
    * glm::rotate(glm::mat4(1.0f), theta,
glm::vec3(0,1,0))
    * glm::translate(glm::mat4(1.0f), -pivot)
    * glm::scale(glm::mat4(1.0f),
glm::vec3(2.0f, 1.0f, 1.0f));
```

Homework: Transformation Mini-Scene

- Build a mini-scene with 3 objects: one spins, one orbits, one shears or non-uniform scales
- Use correct MVP and normal matrix; include a toggled “wrong normals” mode for comparison
- Submit: source code + 3 screenshots + short reflection (≤ 150 words) on matrix order and normals
- **Add interaction:** rotate object angle using keyboard.
- Assessment: correctness, clarity of structure, incremental commits, and visual debugging aids

```
// Suggested structure
mat4 buildModel_ObjectA(float t);
mat4 buildModel_ObjectB(float t);
mat4 buildModel_ObjectC(float t);

// Render loop: set uModel per object + uNormalMat
accordingly
```

